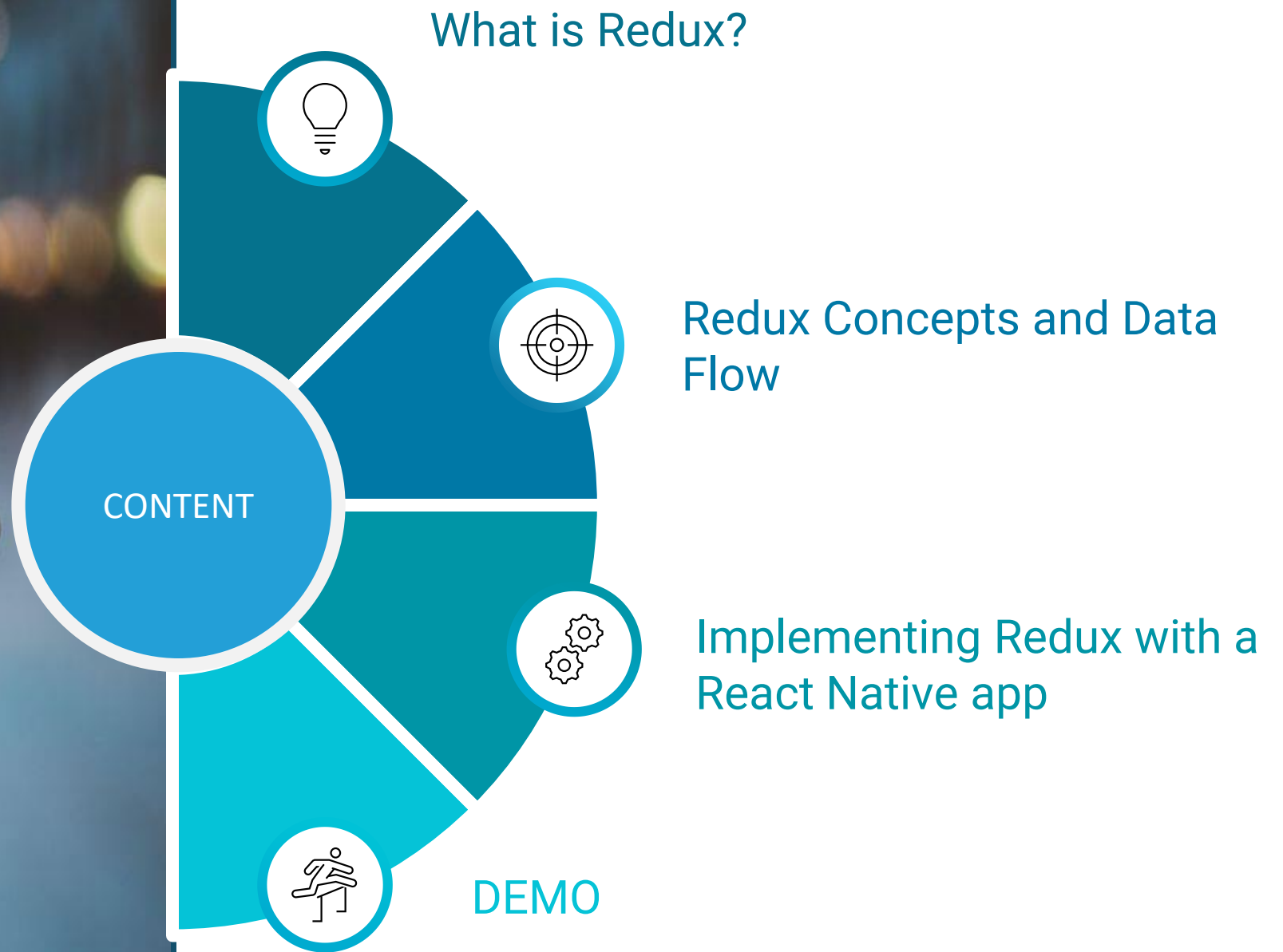


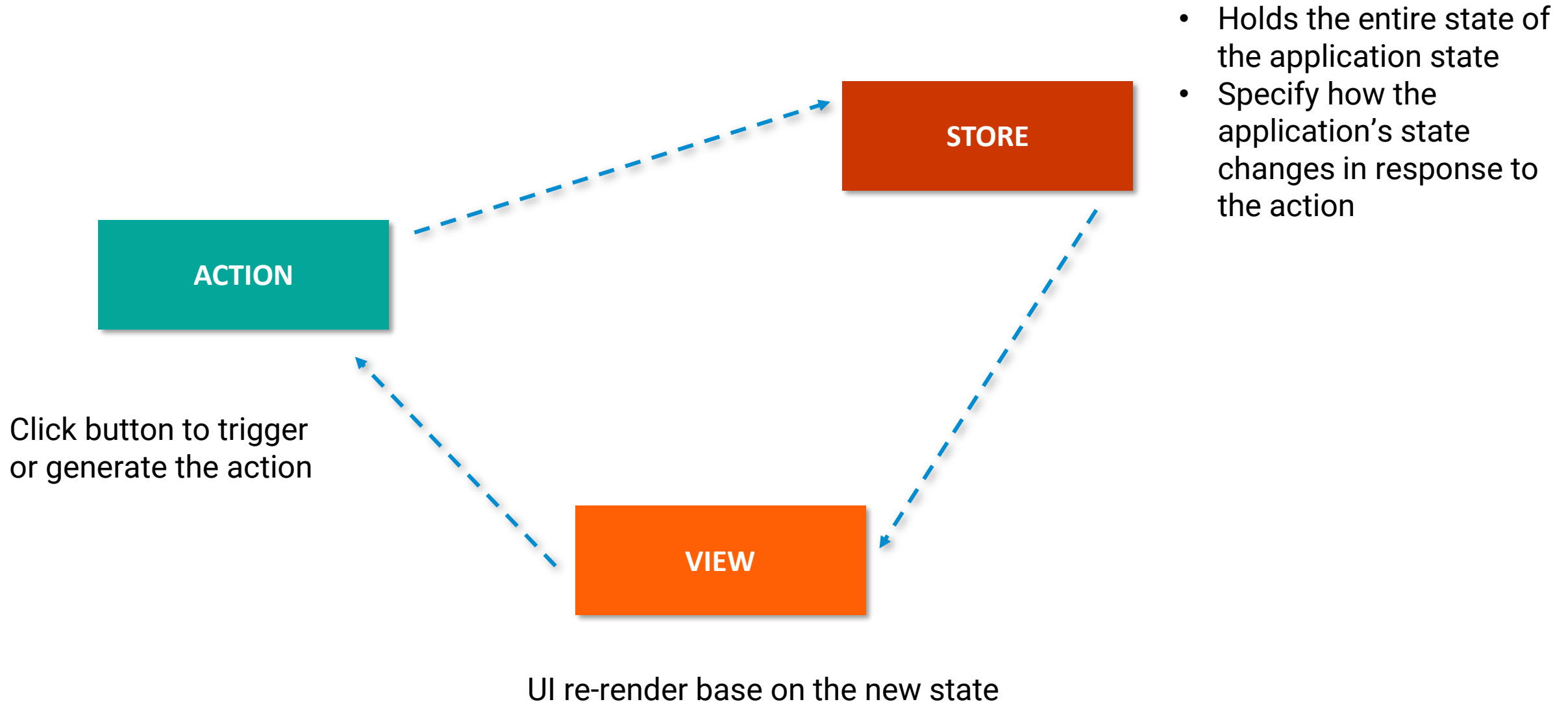




1

WHAT IS REDUX?

- Is a pattern and library for managing and updating application state, using events called "**actions**"
- Is an open-source library made using the scripting language JavaScript
- Allow separating of state (data) management from the presentation and logic layers
- Using Redux in case:
 - App state is updated frequently over time
 - Logic to update that state may be complex
 - App has a medium or large-sized codebase, and might be worked on by many people



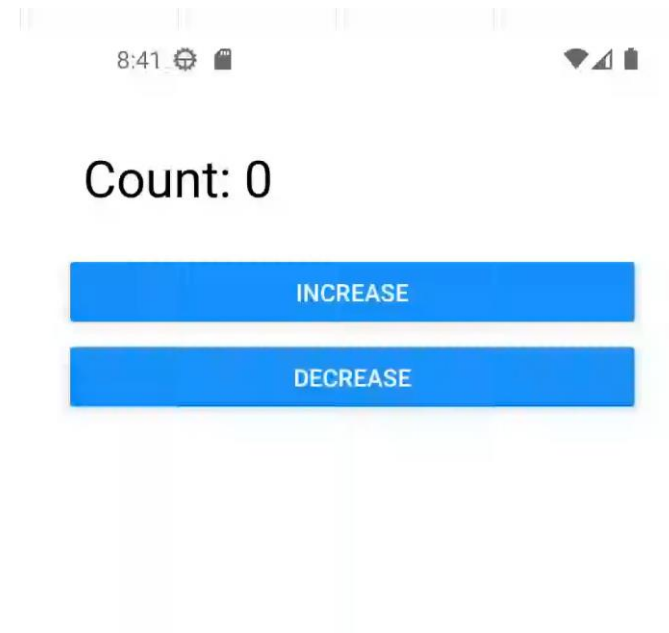
FEATURES OF REACT REDUX

- Reduces code complexity
- Easier Maintainability
- Reduces Time
- Effortless Debugging

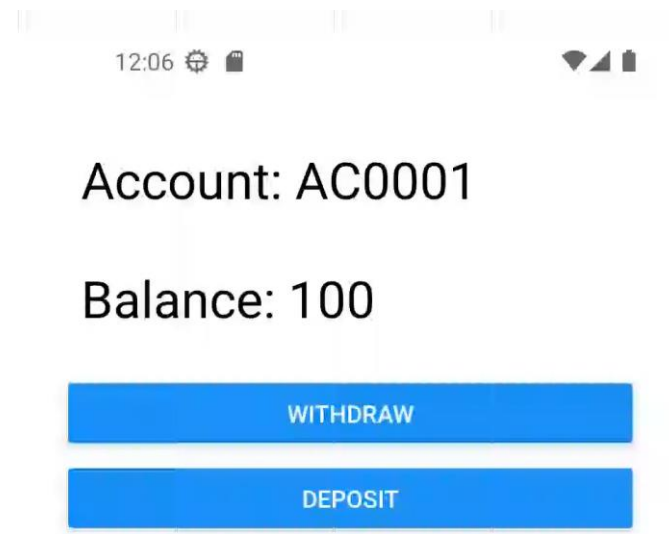
2

REDUX CONCEPTS AND DATA FLOW

➤ Counter App



➤ Bank App



VIEW

- UI - part of application interface
- Displays state of the store

Count: 0

INCREASE

DECREASE

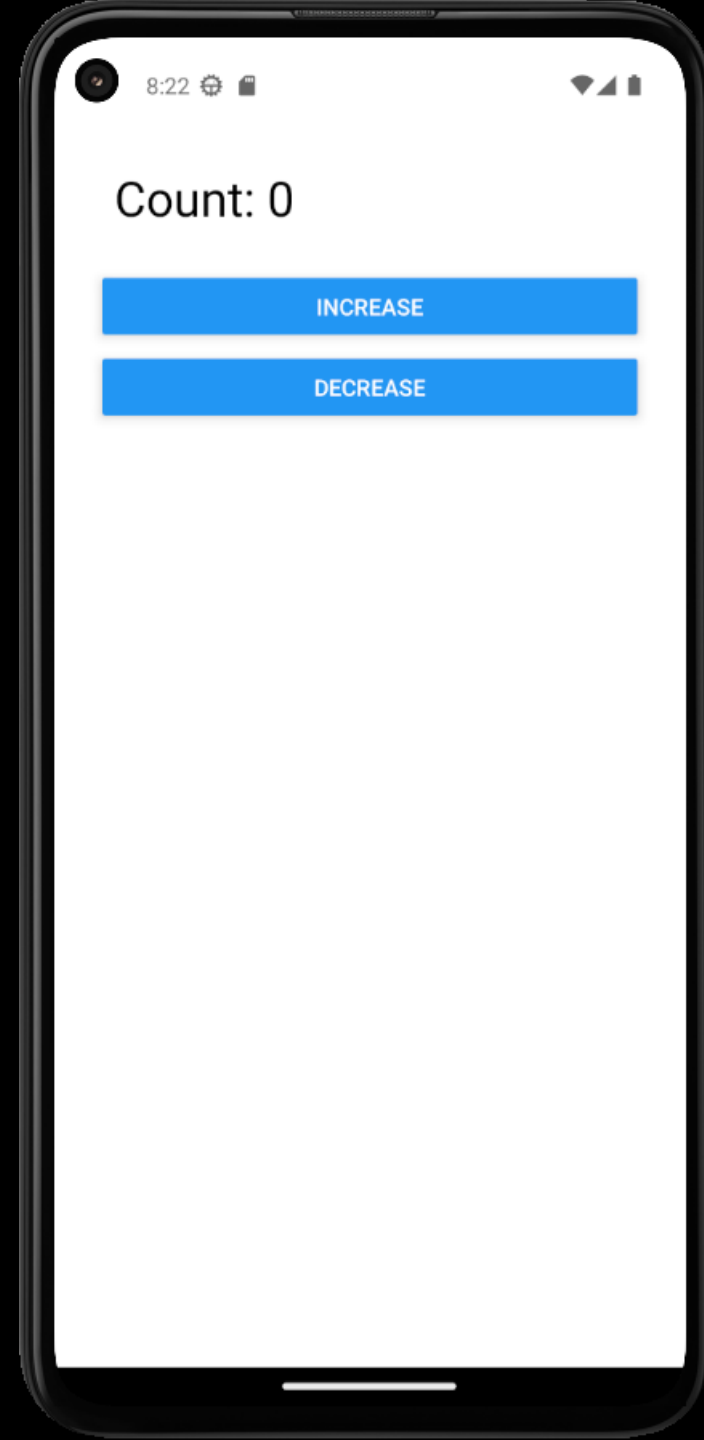
ACTION & ACTION CREATOR

- **Action** are plain JavaScript objects
 - type (mandatory)
 - payload (optional)
- **Action creator** are functions which generate the action objects

```
//action types
export const INCREMENT="INCREMENT"
export const DECREMENT="DECREMENT"
```

```
export const inc = (num)=>{
  return {
    type: INCREMENT,
    payload: num
  }
}
```

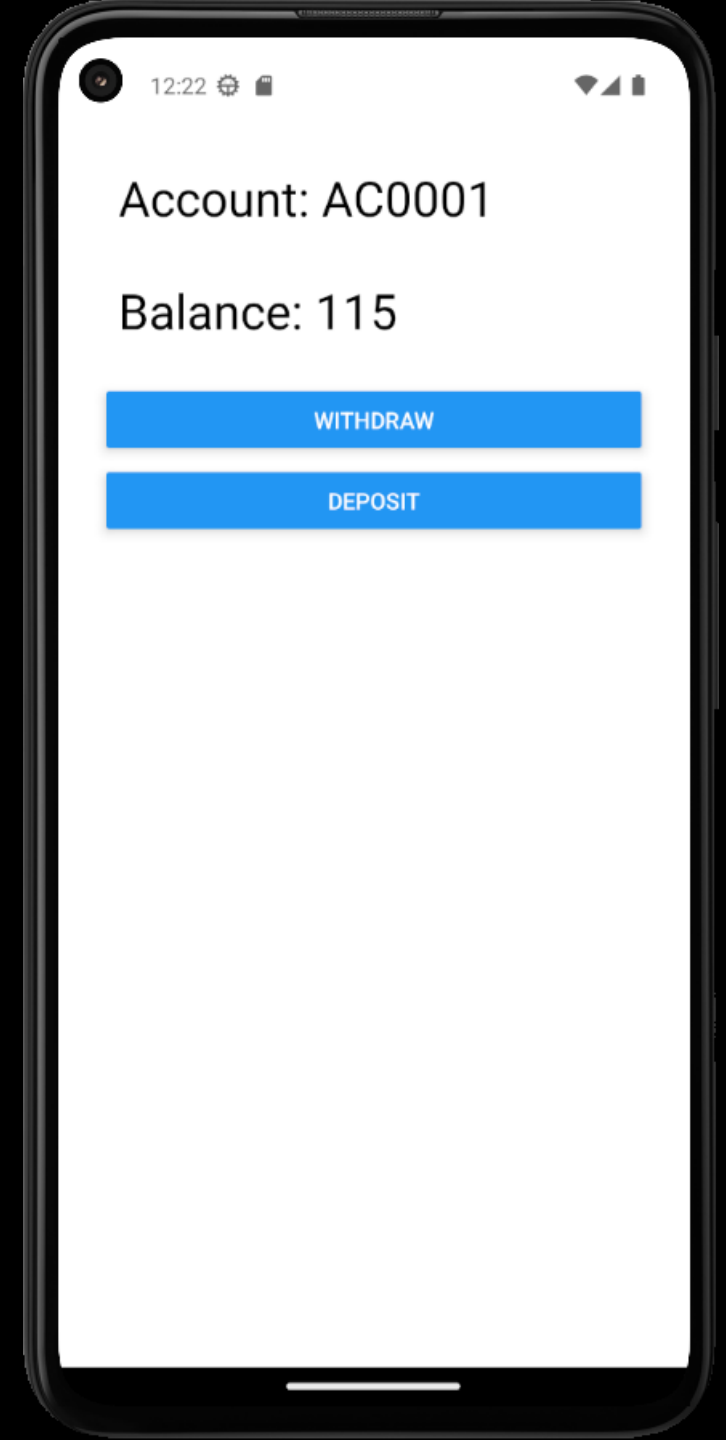
```
export const dec = (num)=>{
  return {
    type: DECREMENT,
    payload: num
  }
}
```



```
export const WITHDRAW = "Withdraw"  
export const DEPOSIT = "Deposit"
```

```
export const Withdraw = (amount) =>{  
  return {  
    type: WITHDRAW,  
    payload: amount  
  }  
}
```

```
export const Deposit = (amount) =>{  
  return {  
    type: DEPOSIT,  
    payload: amount  
  }  
}
```



STATE

- Is application's internal data
- Can be any JavaScript type:
 - Number
 - String
 - Boolean
 - Array
 - Object

Account: AC0001

Balance: 100

WITHDRAW

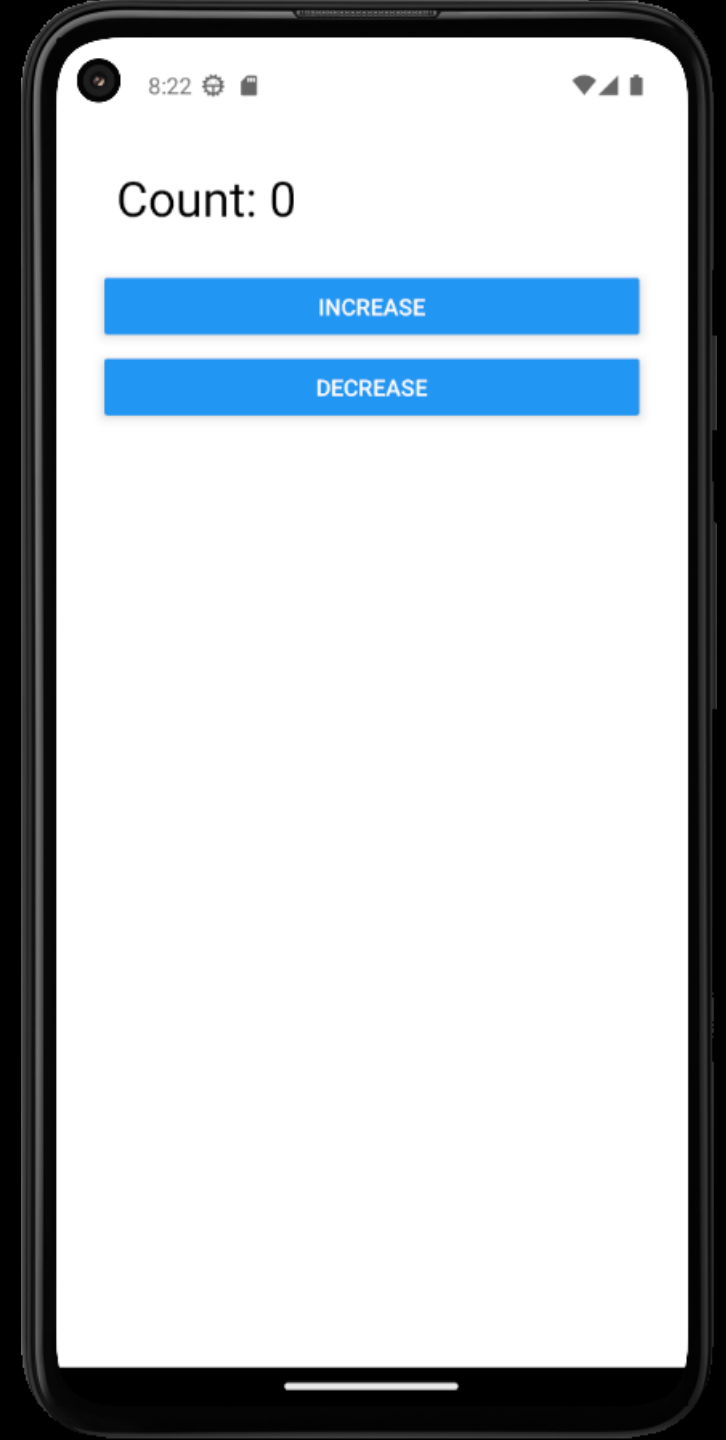
DEPOSIT

```
const initialState= {  
  accountName: 'AC0001',  
  balance: 100  
}
```

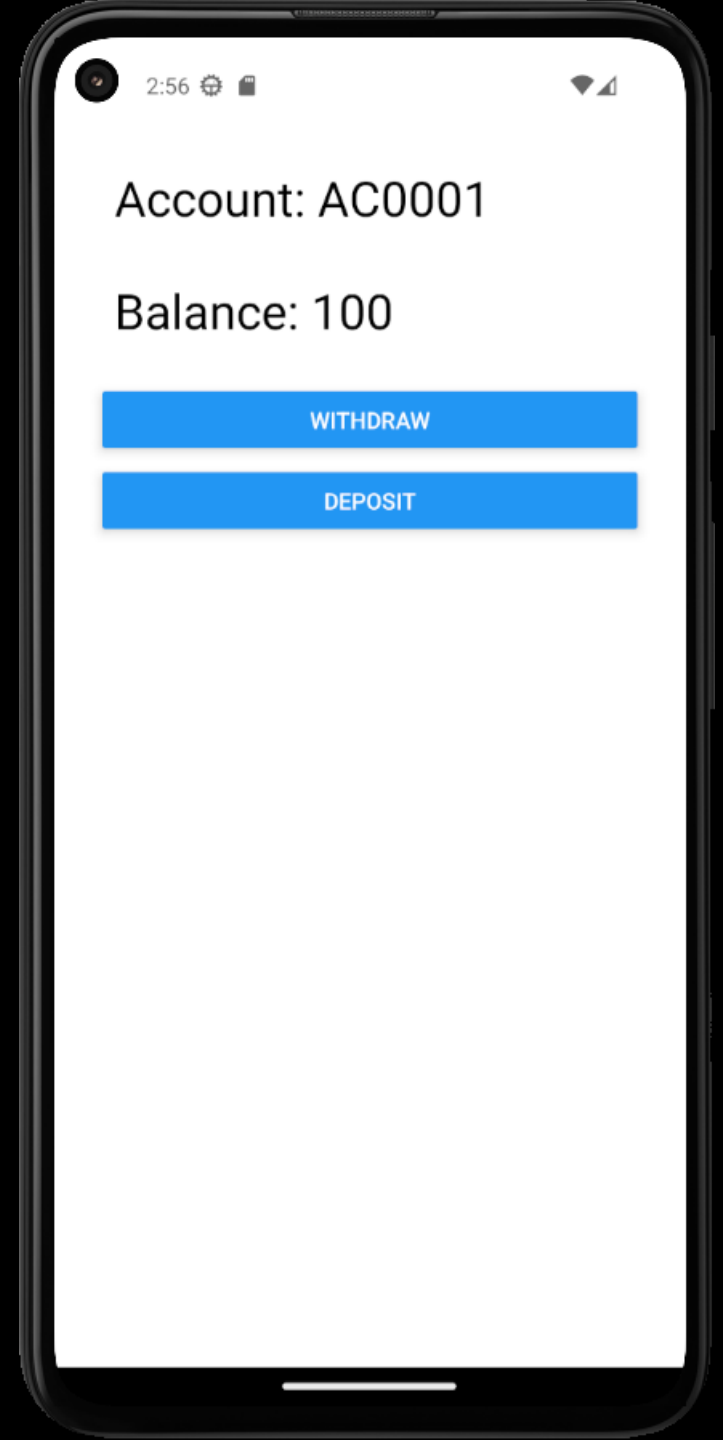
REDUCER

- Are pure functions that handle state logic, accepting the initial state and action type to update and return the state
- Has arguments:
 - current state
 - action

```
const initialState= {  
  count: 0  
}  
  
const counterReducer = (state=initialState, action)=>{  
  switch(action.type){  
    case "INCREMENT":  
      return {  
        count: state.count + action.payload  
      }  
    case "DECREMENT":  
      return {  
        count: state.count - action.payload  
      }  
    default:  
      return state;  
  }  
}  
  
export default counterReducer
```



```
const initialState= {  
  accountName: 'AC0001',  
  balance: 100  
}  
  
const bankReducer = (state=initialState, action)=>{  
  switch(action.type){  
    case "Withdraw":  
      return {  
        ...state,  
        balance: state.balance - action.payload  
      }  
    case "Deposit":  
      return {  
        ...state,  
        balance: state.balance + action.payload  
      }  
    default:  
      return state;  
  }  
}  
export default bankReducer
```



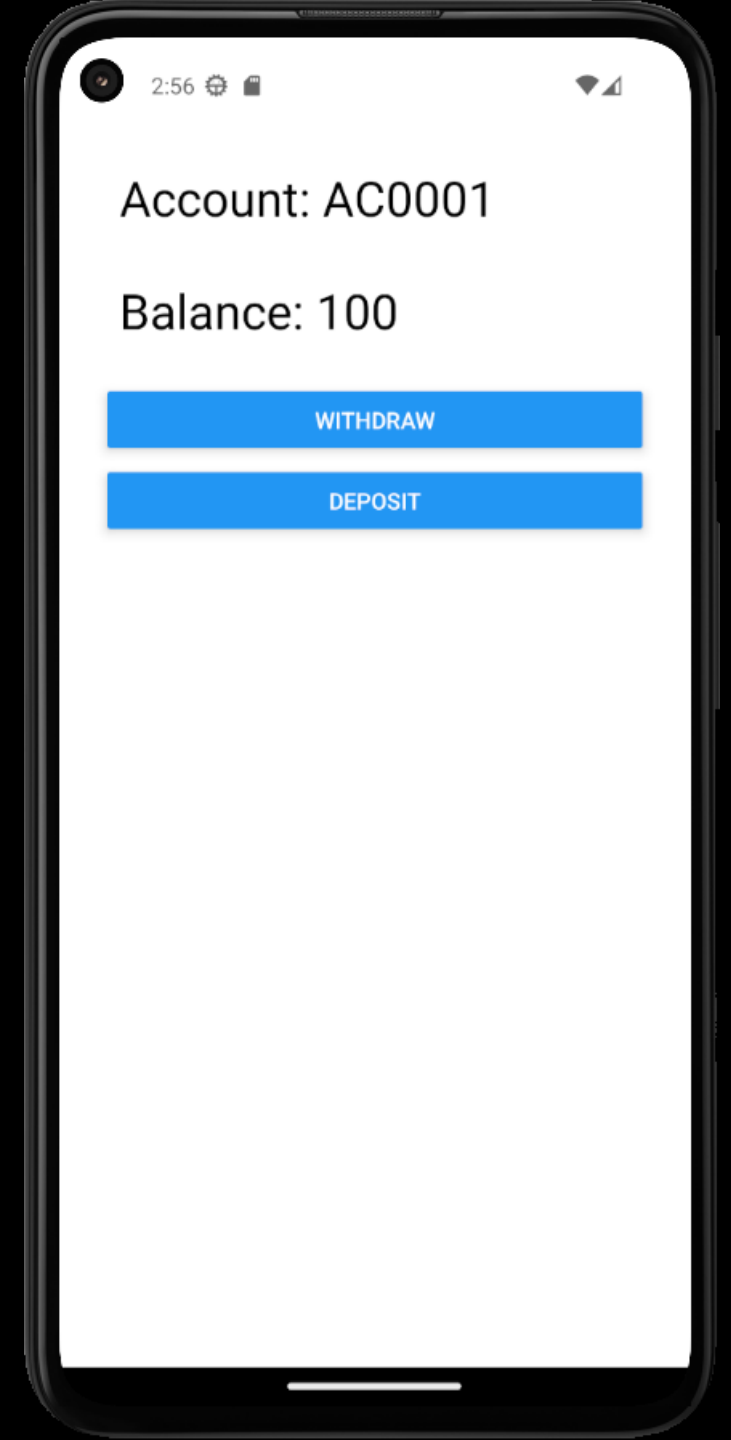
STORE

- Redux has a **single store** that holds the application information
- Receive actions, pass them through all the registered middleware, and then use reducers to calculate a new state and save it
- When it receives an action, the store will notify all the registered listeners that a change to the state has been made. This will allow various parts of the system, like the UI, to update themselves according to the **new state**

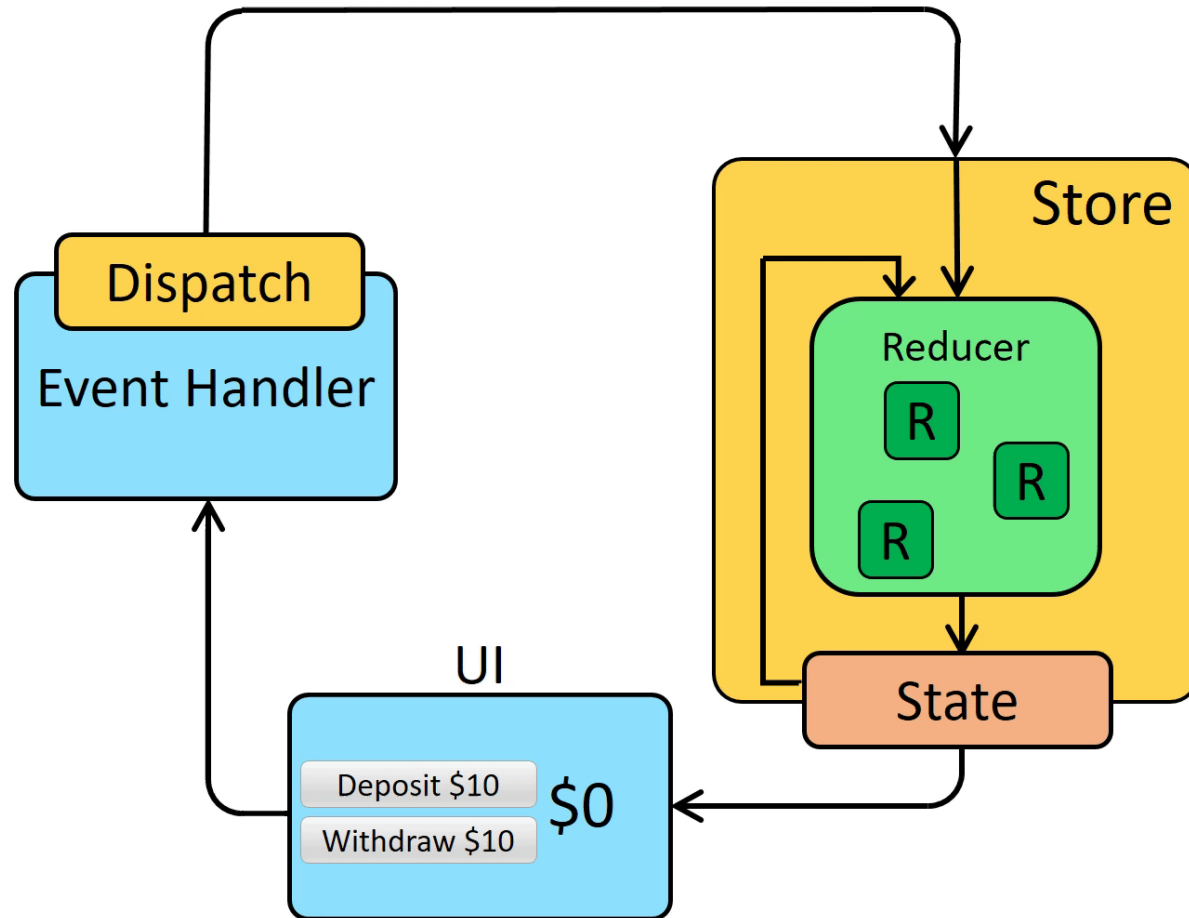
```
import bank from "../bankReducer"
import { combineReducers } from "redux";
import { createStore } from "redux";

const allReducers = combineReducers({
  bank
});

export const store = createStore(
  allReducers,
);
```



ONE WAY DATA FLOW IN REDUX

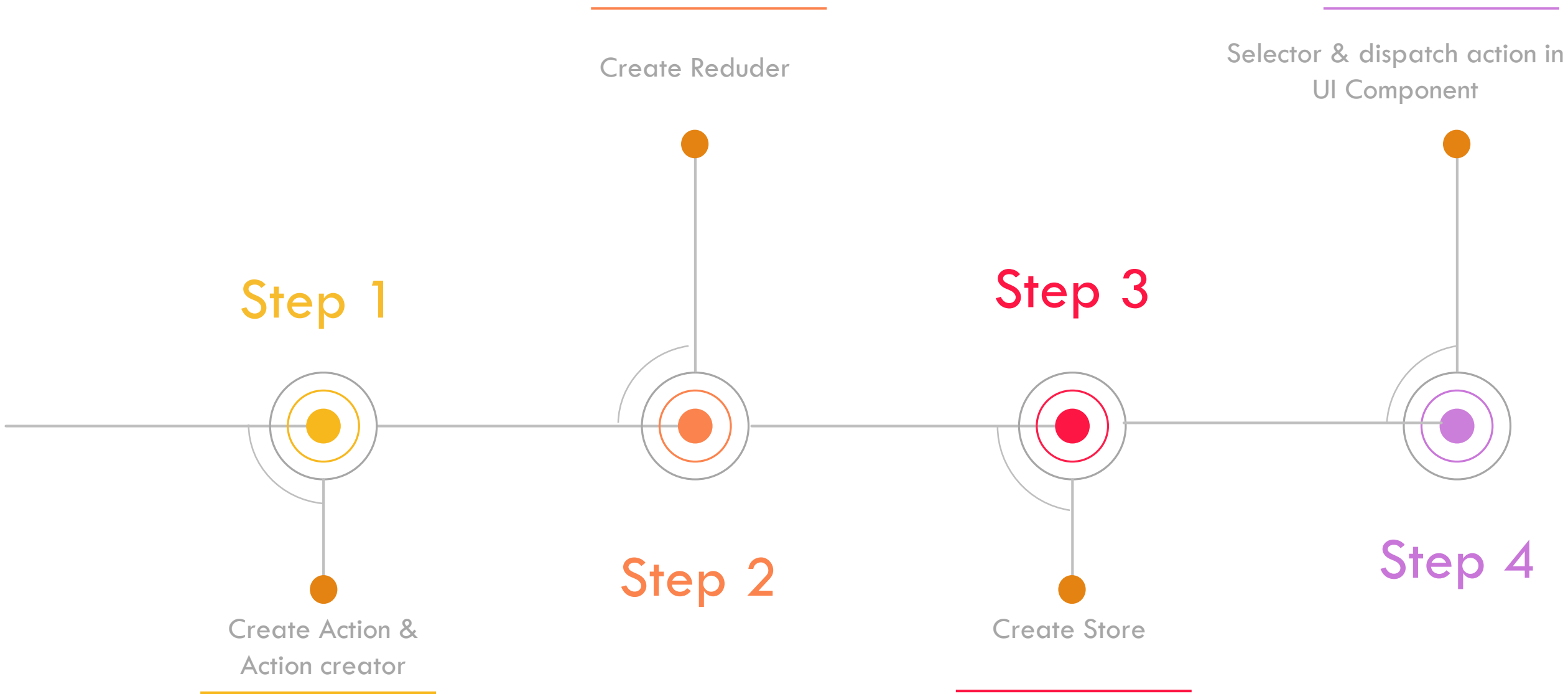


3

IMPLEMENTING REDUX WITH A REACT NATIVE APP

Install Library

- npm install **redux**
- npm install **react-redux**
- npm install **redux-thunk**
- npm install **@react-native-async-storage/async-storage**



Step 1

Create Action &
Action creator

Create Reducer

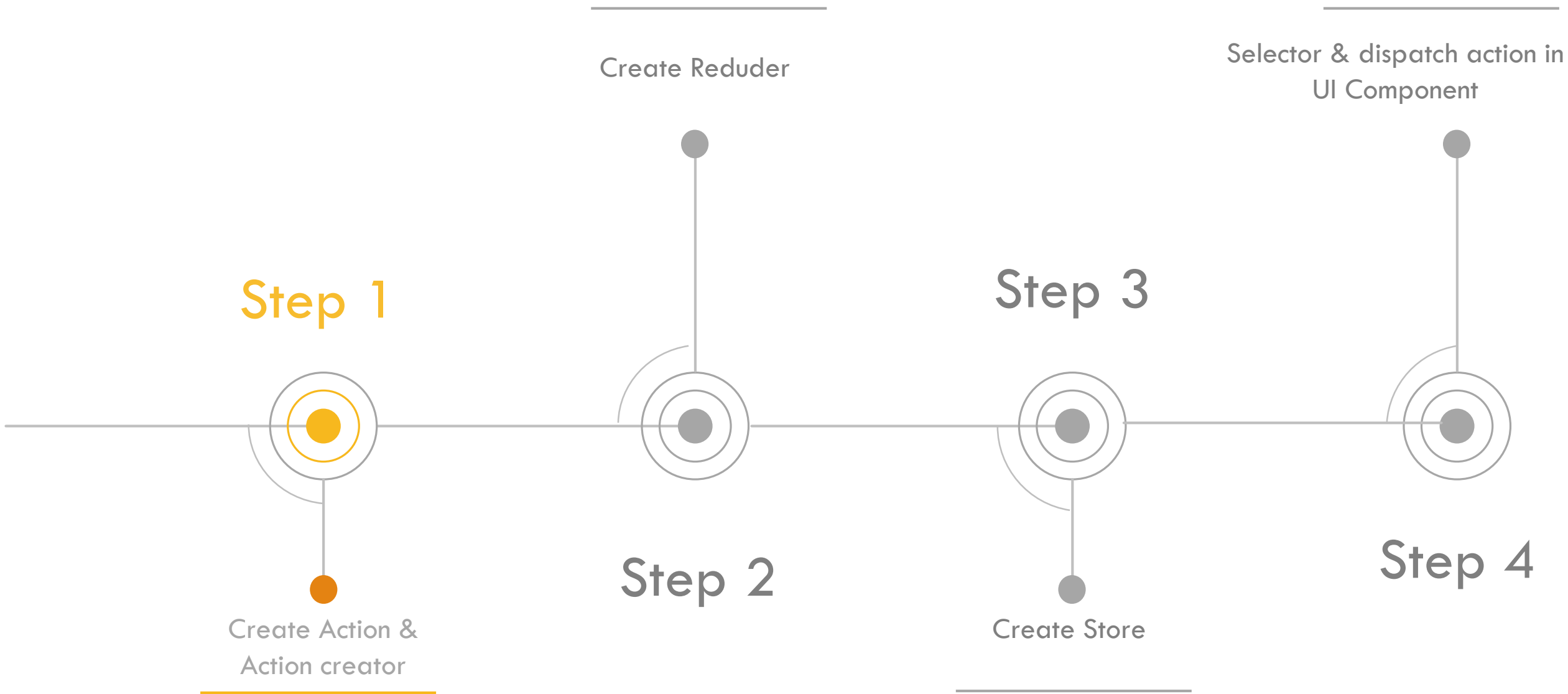
Step 2

Step 3

Create Store

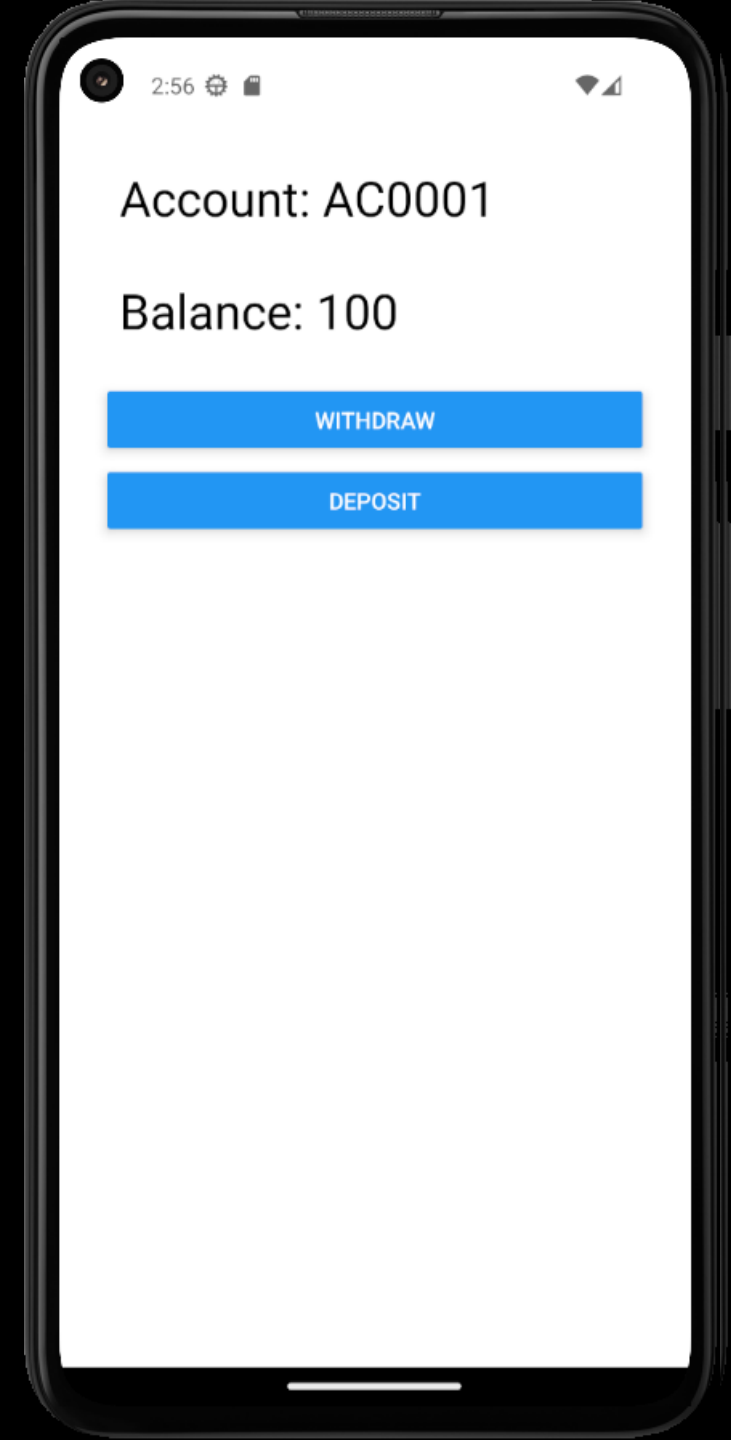
Selector & dispatch action in
UI Component

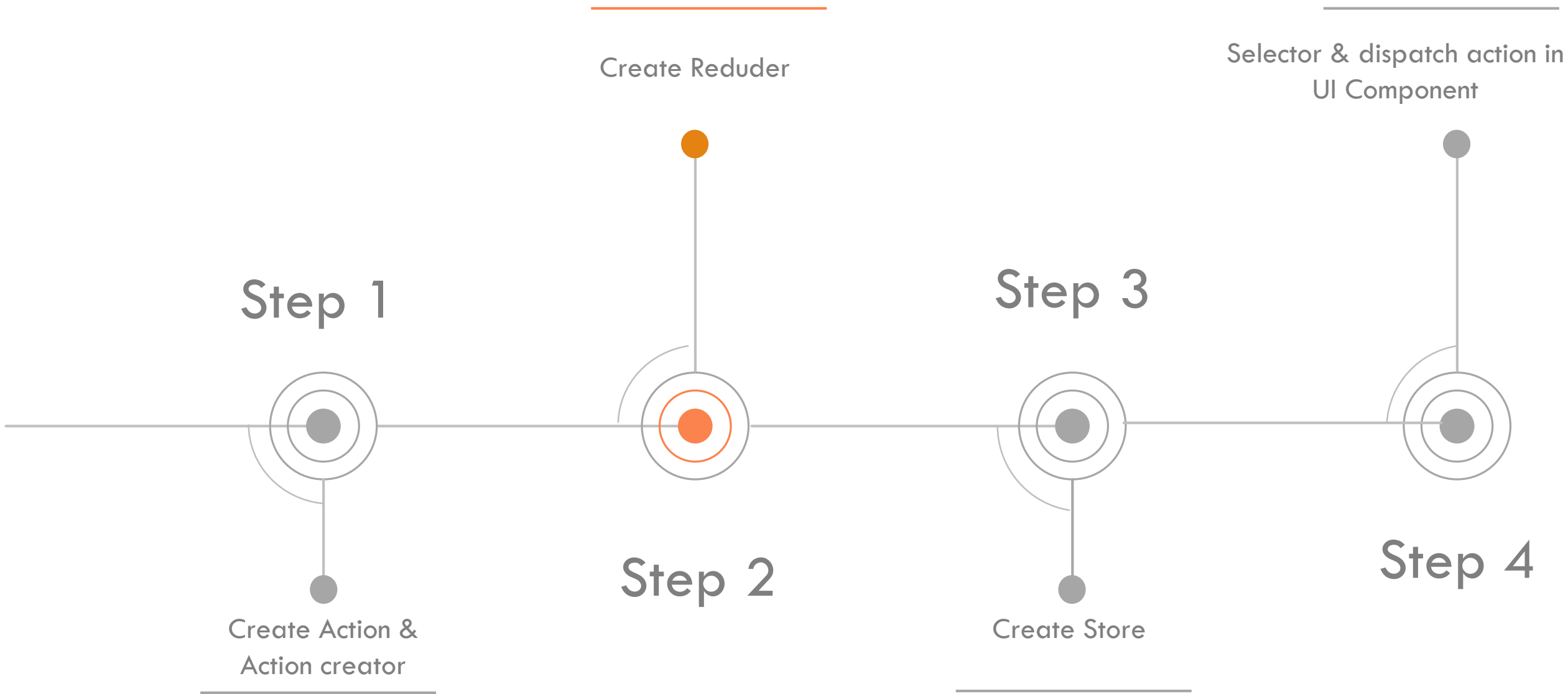
Step 4



```
export const WITHDRAW = "Withdraw"
export const DEPOSIT = "Deposit"

export const Withdraw = (amount) =>{
  return {
    type: WITHDRAW,
    payload: amount
  }
}
export const Deposit = (amount) =>{
  return {
    type: DEPOSIT,
    payload: amount
  }
}
```





```
const initialState= {
  accountName:'AC0001',
  balance: 100
}

const bankReducer = (state=initialState, action)=>{

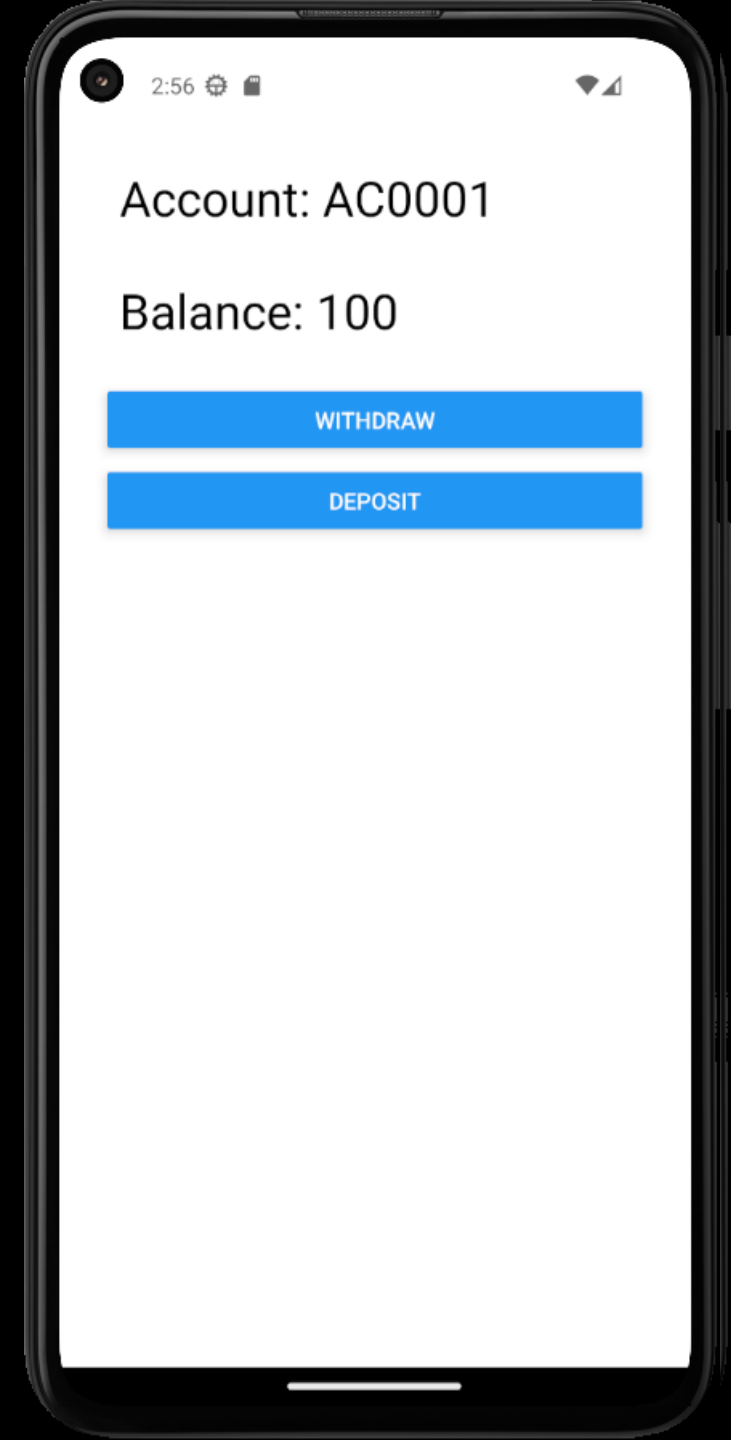
  switch(action.type){

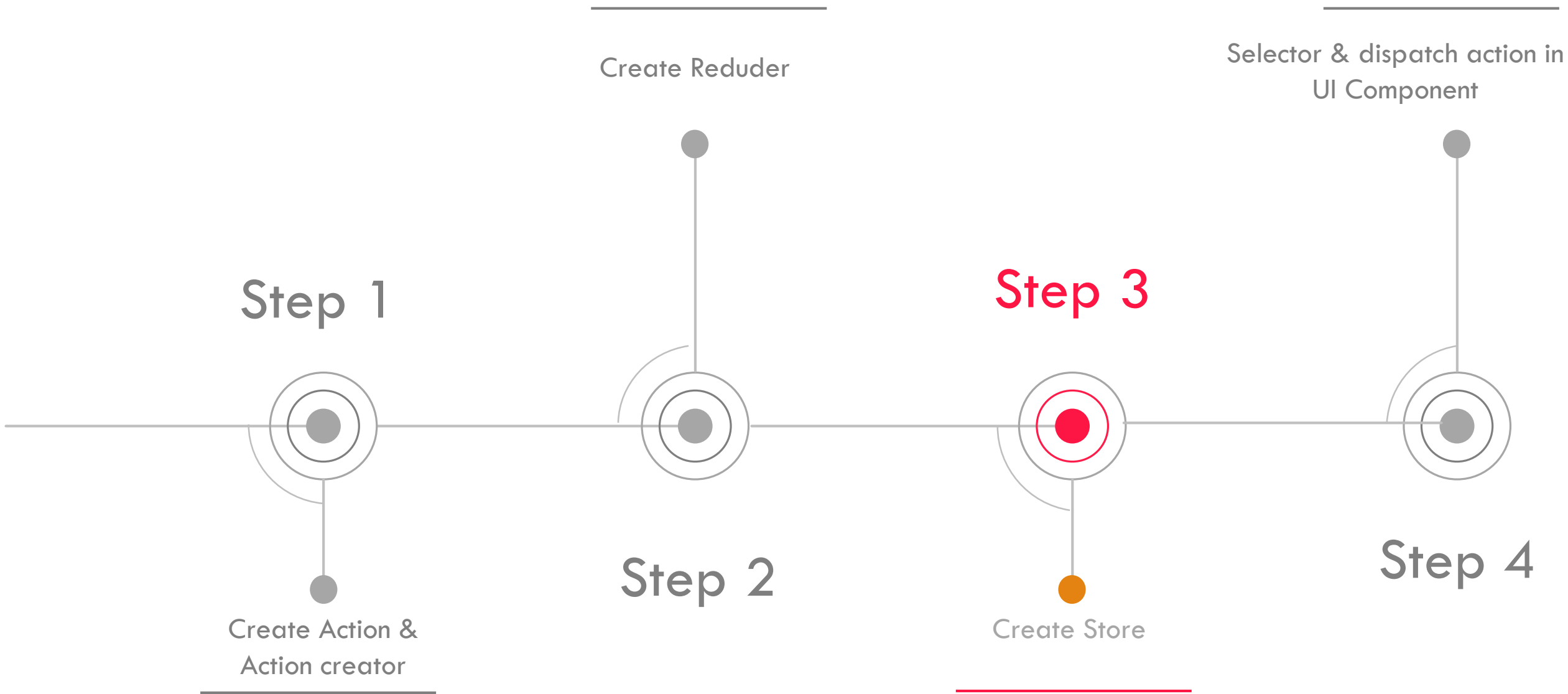
    case "Withdraw":
      return {
        ...state,
        balance: state.balance - action.payload }
    case "Deposit":
      return {
        ...state,
        balance: state.balance + action.payload }

    default:
      return state;
  }

}

export default bankReducer
```





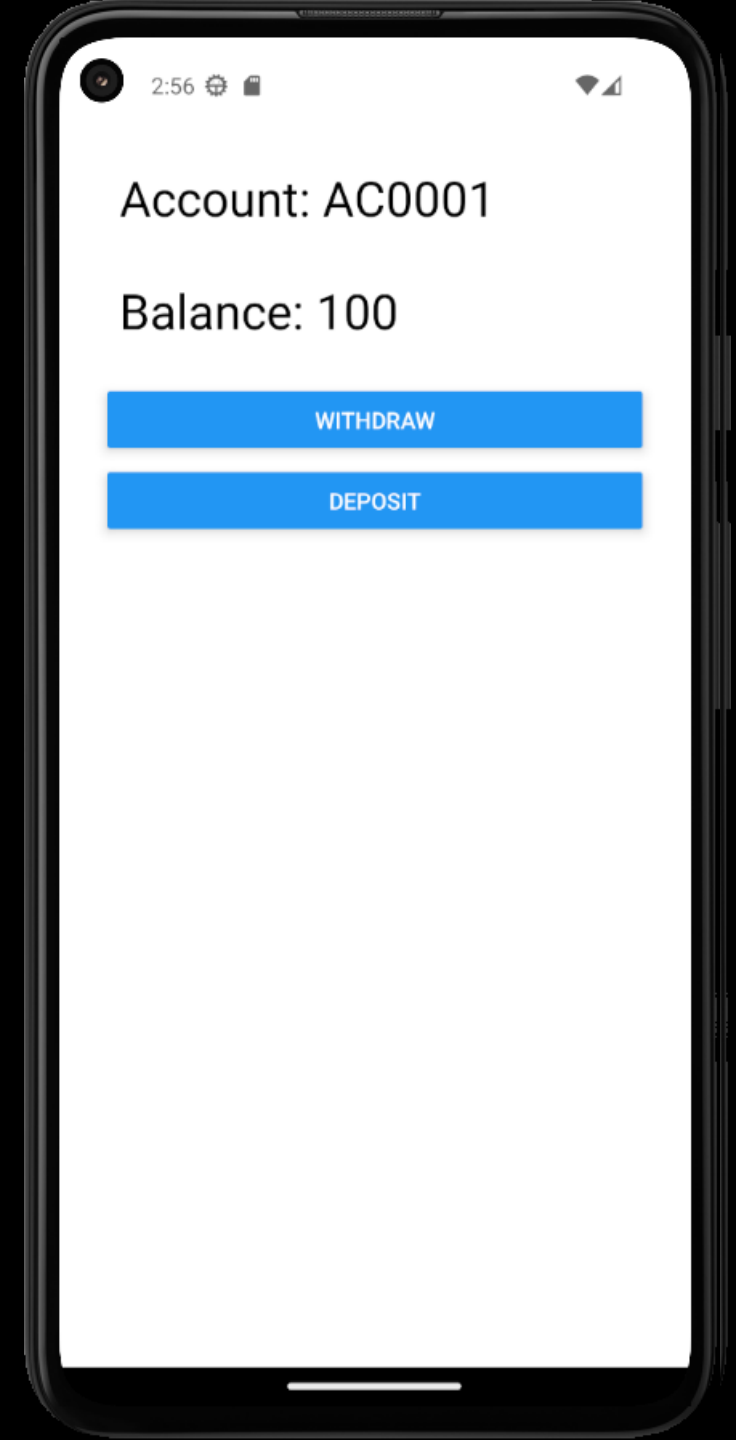
```
import counter from "../counterReducer"
import bank from "../bankReducer"
```

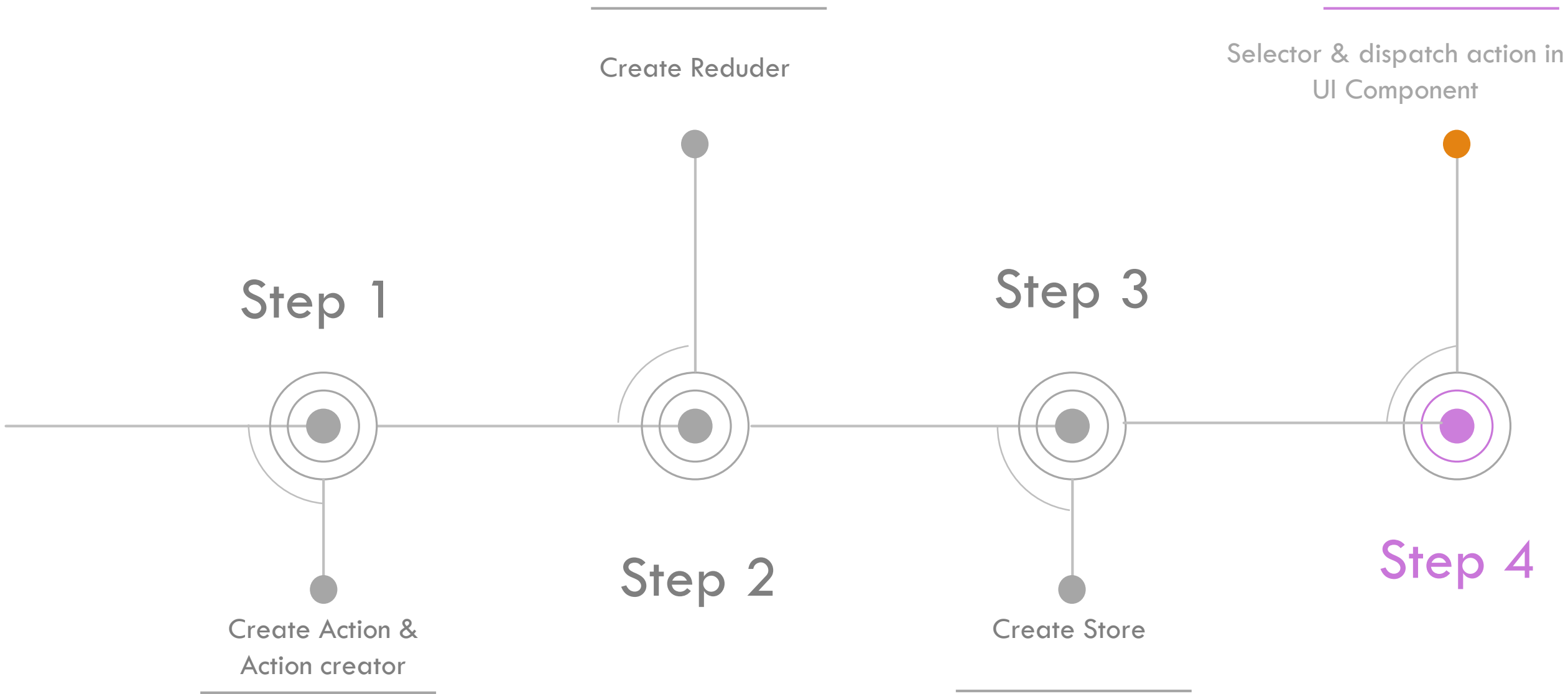
```
import { combineReducers } from "redux";
import { createStore } from "redux";
```

```
const allReducers = combineReducers({
  counter,
  bank
```

```
});
```

```
export const store = createStore(
  allReducers,
);
```



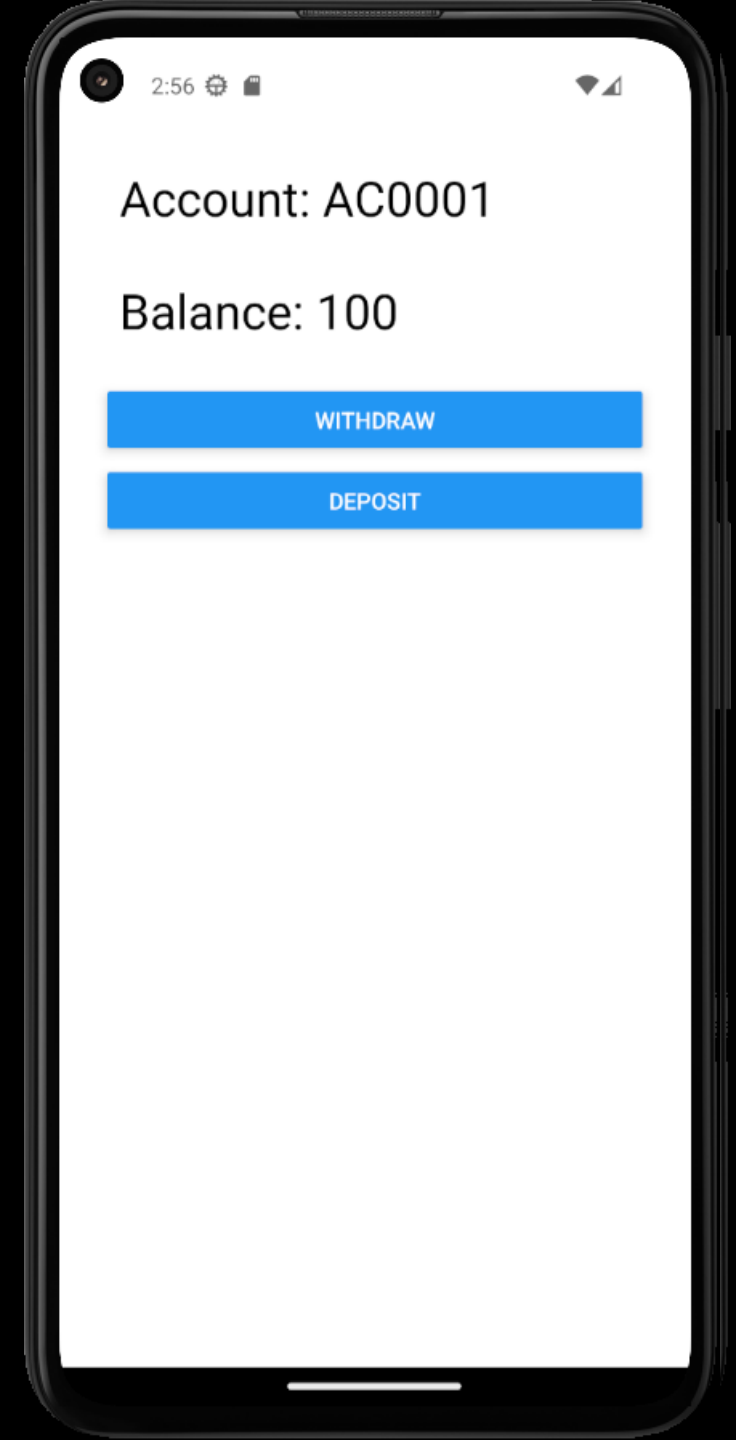


```
import { useSelector } from "react-redux";  
import { useDispatch } from "react-redux";
```

```
import {Withdraw, Deposit } from "../redux/bankAction";  
const MyBankView = () => {
```

```
  const bank = useSelector((state) => state.bank);  
  const dispatch = useDispatch();
```

```
  return (  
    <View style={styles.container}>  
      <Text style={styles.title}> Account:  
        {bank.accountName} </Text>  
      <Text style={styles.title}> Balance:  
        {bank.balance} </Text>  
      <Button title="Withdraw"  
        onPress={() => dispatch(Withdraw(5))} />  
      <View style={styles.separate} />  
      <Button title="Deposit"  
        onPress={() => dispatch(Deposit(10))} />  
    </View>  
  )  
}  
export default MyBankView
```



```
import { Provider } from "react-redux";

import MyBankView from '../components/MyBankView';
import {store} from '../redux/index'

export default function App() {
  return (

    <Provider store={store}>

      <View style={styles.container}>
        <MyBankView />
      </View>

    </Provider>

  )
}
```

